



DEEP LEARNING ESSENTIALS

INNOVATIVE ASSIGNMENT – II

DENSENET VS CONVENTIONAL CNN FOR IMAGE CLASSIFICATION

PROJECT REPORT

(Deep Learning / Image Classification)

Application: Image Classification

Dataset: CIFAR-10

Submitted By: GAUTHAMAN V

Department: CSE(AIML)

BATCH: 8

Academic Year: III YEAR / SECTION: A

1. Problem Statement

Implement DenseNet and a conventional CNN for the same image-classification dataset. Compare parameter usage, feature reuse and classification performance and analyze the effect of dense connectivity.

The project uses the CIFAR-10 image-classification dataset because the assignment permits the use of an alternative dataset. The same dataset and evaluation setup are used for both models so that the comparison focuses on the architectural difference between a conventional CNN and DenseNet.

2. Objective

- Build a small conventional convolutional neural network for CIFAR-10 classification.
- Implement a compact DenseNet with dense blocks and transition layers.
- Use identical dataset preparation and training conditions for both models.
- Calculate and compare total and trainable parameter counts.
- Demonstrate feature reuse through DenseNet dense connectivity.
- Compare classification performance using accuracy, loss and confusion matrices.
- Analyze the practical effect of dense connectivity on the two models.

3. Concept Used

Image classification: assigning an input image to one of a fixed set of classes. CIFAR-10 contains 10 classes such as airplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck.

Convolution: a convolutional layer applies learnable filters to local image regions to extract useful spatial patterns such as edges, textures and shapes.

Conventional CNN: convolutional layers are arranged mainly in a sequential chain, so a layer normally receives the output of the preceding layer.

DenseNet: within a dense block, each layer receives the feature maps produced by all preceding layers through concatenation. This encourages feature reuse and improves information flow.

Parameter: a learnable weight or bias adjusted during training. Parameter count is used to compare model size and learning capacity.

4. Methodology / Working Steps

- Dataset preparation: download CIFAR-10 through torchvision and use a reproducible train/validation/testsetup.
- Preprocessing: convert images to tensors and normalize them using the same transformation for both models.
- Conventional CNN: train a compact sequence of convolution, activation and pooling stages followed by a classifier.
- DenseNet: train a compact architecture containing dense blocks, feature-map concatenation and transition layers.
- Controlled experiment: keep optimizer, learning rate, batch size, epoch count, random seed and evaluation procedure the same.
- Evaluation: calculate parameter counts, accuracy, loss and confusion matrices for both models.

- Feature reuse analysis: inspect the dense-block connectivity and representative intermediate feature maps.
- Comparison: relate the measured results to the effect of dense connectivity.

Architecture / Flow

CIFAR-10 Dataset	Same data → same preprocessing
Conventional CNN	Sequential convolutional feature extraction
DenseNet	Dense blocks + concatenated feature maps
Training & Evaluation	Same training conditions
Comparison	Parameters • Feature Reuse • Accuracy • Loss
Analysis	Effect of Dense Connectivity

Conventional CNN connectivity

Input → Conv Block 1 → Conv Block 2 → Conv Block 3 → Classifier

DenseNet connectivity

Input → Dense Block → Transition → Dense Block → Global Average Pooling → Classifier

Inside a dense block, earlier feature maps are concatenated with newly generated features. This direct reuse is the central architectural difference studied in the project.

5. Implementation

5.1 Tools & Libraries

Tool / Library	Use
Python	Core programming language
PyTorch	Neural-network implementation and training
Torchvision	CIFAR-10 dataset loading and image transformations
NumPy	Numerical operations and result handling
Matplotlib	Training curves and visualizations
Scikit-learn	Confusion matrix and classification analysis
Jupyter Notebook	Interactive step-by-step analysis
uv	Python environment and dependency management

5.2 Project Structure `densenet-vs-cnn/`

```

■■■ README.md
■■■ requirements.txt / pyproject.toml

```

```

■■■ dataset/
■ ■■■ README.md
■■■ src/
■ ■■■ models.py
■ ■■■ dataset.py
■ ■■■ train.py
■ ■■■ evaluate.py
■ ■■■ compare.py
■■■ notebooks/
■ ■■■ model_analysis.ipynb
■■■ results/
■ ■■■ training_curves.png
■ ■■■ accuracy_comparison.png
■ ■■■ parameter_comparison.png
■ ■■■ confusion_matrix_cnn.png
■ ■■■ confusion_matrix_densenet.png
■ ■■■ model_comparison.csv
■■■ screenshots/
■■■ report/

```

5.3 Source Code / GitHub Repository

Repository: <https://github.com/runeking2006/Image-Classification-Analysis>

The source code is organized so that model definitions, dataset handling, training, evaluation and comparison are separated into understandable modules.

6. Results & Output

The completed experiment is intended to generate the following measurable outputs for the same CIFAR-10 test set:

Output	Purpose
Parameter count	Compare total and trainable parameters of the two models.
Training / validation curves	Observe learning behaviour and convergence.
Test accuracy	Compare final classification performance.
Confusion matrices	Identify class-wise strengths and errors.
Feature-map examples	Visually inspect intermediate learned representations.
Comparison table	Summarize the measured differences between CNN and DenseNet.

7. Analysis

The central comparison is architectural. A conventional CNN passes information through a mainly sequential chain of convolutional blocks. DenseNet instead provides direct access to earlier feature maps inside each dense block through concatenation. This makes feature reuse an explicit part of the architecture.

Parameter usage should be interpreted together with the architecture rather than as a simple “more connections means more parameters” rule. DenseNet uses a growth rate so that each layer contributes a controlled number

of new feature maps while reusing previously generated representations. The experiment therefore measures the actual parameter counts instead of assuming which model must be smaller.

Classification performance is compared under identical training conditions. Accuracy and loss show overall behaviour, while confusion matrices help identify classes that remain difficult for either model. Intermediate feature maps provide a qualitative view of how representations evolve.

Important: numerical metrics and output screenshots are deliberately not fabricated in this report. They must come from the verified project run so that the report remains consistent with the actual source code and GitHub repository.

8. Conclusion

This project studies the effect of dense connectivity by implementing a conventional CNN and a compact DenseNet for the same CIFAR-10 image-classification task. Both models are evaluated using the same experimental conditions, allowing their parameter usage, feature reuse and classification performance to be compared directly.

The main conceptual difference is that the conventional CNN mainly forwards features sequentially, whereas DenseNet concatenates feature maps from preceding layers within a dense block. This encourages reuse of previously learned representations and provides stronger information flow between layers.

The final conclusion will be based on the measured parameter counts and classification results produced by the implementation. The project therefore demonstrates not only how DenseNet is constructed, but also how its connectivity pattern changes the behaviour of a convolutional classification model.

9. References

- Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). Densely Connected Convolutional Networks. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
- Krizhevsky, A. (2009). Learning Multiple Layers of Features from Tiny Images. Technical Report, University of Toronto. CIFAR-10 dataset.
- PyTorch Documentation – Neural network modules, training and tensor operations.
- Torchvision Documentation – CIFAR-10 dataset and image transformation utilities.
- Scikit-learn Documentation – Confusion matrix and classification evaluation utilities.
- Matplotlib Documentation – Plotting and visualization utilities.